

Part A

Assumptions

1. Documents are mostly clean PDF/DOCX not noisy or scanned documents, average 10-15 page , English only.
2. Model weights available locally in the serving layer.
3. API gateway already handles auth and rate limits.
4. No per user Access Control List (ACL)

Architecture

An Retrieval-Augmented Generation (RAG) LLM.

- **Components:** PostgreSQL + pgvector, vLLM, FastAPI.
- **Ingestion pipeline executed monthly:** Parsing of documents > chunking > attach metadata > embed with embedding model > build index > golden-set eval -> atomic swap.
- **Serving pipeline:** User query -> hybrid retrieval (BM25 + vector, merged by reciprocal rank fusion (RRF), top 50) -> cross-encoder re-rank to top-5 -> inject context into System Prompt -> stream from vLLM.

The architecture uses FastAPI as an orchestrator to coordinate data flow between a PostgreSQL + pgvector database and vLLM for low-latency model inference. During ingestion, a monthly off-peak scheduled pipeline parses, chunks, hashes, embeds, and evaluates documents before executing an atomic database swap.

When serving queries, FastAPI executes retrieval and reranking, and injects the top context chunks into the system prompt that includes instructions to cite document and line in which information is retrieved from alongside the user's question, it then forwards the prompt to vLLM and streams the generated tokens directly back to the user.

Key decisions and trade-offs

Flat vector search vs Approximate Nearest Neighbor (ANN)

- 2,000 documents chunked is relatively small. Exact flat search over small volume is fast. ANN methods introduces index build overhead and hyperparameter tuning. Caveat is that flat search scales linearly with context volume, if corpus grows significantly query latency will increase and will be worth migration to ANN search algorithms to maintain speed.

Full Rebuild vs Incremental updates

- Incremental updates save ingestion compute time but in-place updates require tracking modified/deleted document states and be harder to maintain. At relatively small-scale of 2000 docs a monthly full rebuild is relatively fast and less risk in database maintenance.

Postgres + pgvector vs Dedicated DB

- Less maintenance for single engineer, but if documents grow significantly, worth migrating to dedicated DB like Qdrant.

Cross-encoder re-rank

- Increases retrieval precision. Top-5 relevant chunks instead of more less-relevant chunks reduces prompt length, saves KV-cache memory and lowers Time-To-First-Token (TTFT). However, Re-ranking adds some extra compute step introducing some overhead.

Streaming in vLLM

- Key component to “snappy” feeling, more of a UX consideration but streaming outputs has a lower TTFT. If content filtering is required, harder to implement.

Monitor post-deployment:

- Performance Metrics: End-to-end latency, TTFT, incoming queue length, active batches, KV-cache pre-emption rate, GPU utilization, error rate.
- Quality Metrics: Distribution of similarity scores over time, "quality" of answers (faithfulness, answer relevance, context precision, and recall).

One failure mode that only surfaces in production

Queries from users could be out of scope or uncovered in the corpus, retrieval could get the next closest thing and the model then attempts to synthesize an answer from irrelevant context, confidently generating hallucinations cited from wrong documents.

Part C

First thing I would investigate would be to check if there is **Stale Indexing**. Since the source documents are correct and up-to-date, the ingestion pipeline might have failed in atomic table swaps or missed file updates. Check the hashes for up-to-date files on the source documents and compare them directly to the document hash columns in Postgres.

If no failure is found here, next I would investigate possible **Retrieval and Reranking Failure**. Do a Retrieval-Augmented Generation Assessment (RAGAS) on the failing queries to calculate exact Context Precision and Context Recall scores. Having the raw Reciprocal Rank Fusion (RRF) scores alongside the Cross-Encoder output logits to see if the re-ranker model is misranking the updated chunks.

I will next investigate to see if problem is caused by **Poor Chunking**, too large chunking could lead to dilution of information and introduce noise as irrelevant information, too small results in missing context. Perform RAGAS evaluation on the failing queries with varying chunking strategy. A low context recall score could indicate important context are being truncated low Context Precision could indicate that chunks are too long diluting the context for generation.